Justificación de las pruebas de la API de avistamientos

Taller de pruebas unitarias y de integración, ISW-2, Sesión 22 (21 de octubre de 2026). Autor: Hernando Javier García Mogollón. Repositorio: https://github.com/Nandoski666/ApiRestFuncional

Resultado: 51 pruebas que pasan: 33 unitarias en `test/unit` y 18 de integración en `test/integracion`. Se ejecutan con `npm test` y la cobertura sale con `npm run cobertura`. Las herramientas son Jest 30 para las pruebas y los dobles, y Supertest 7 para HTTP.

1. Qué cambió en el código

La API del 29 de septiembre validaba con funciones sueltas (`src/validacion.js`). El controlador hacía todo: leía el id, validaba, llamaba al repositorio y decidía el 404. No existía la regla de longitud de la especie ni la de la fecha futura. Los cambios fueron estos:

Cambio	Archivo	Por qué hace la API más testeable
La regla de negocio pasa a la clase `ValidadorDeAvistamiento`, que devuelve `[{ campo, mensaje }]`	`src/validador.js`	Se prueba sin Express ni base de datos. Su salida dice qué campo falló, así que es observable.
El reloj se envuelve en `RelojDelSistema` y el validador lo recibe por el constructor	`src/reloj.js`	Elimina el no determinismo. En las pruebas se le entrega el stub `RelojFijo` y "hoy" es siempre 2026-10-21, lo que da controlabilidad.
Se crea la capa de servicio `ServicioDeAvistamientos`, que recibe el repositorio y el validador por el constructor	`src/servicio.js`	Aplica la inversión de dependencias (DIP). El servicio depende de "algo con `crear`, `buscarPorId`..." y en las pruebas recibe un fake en memoria.
Errores de dominio `ErrorDeValidacion` y `NoEncontrado`, y un controlador que solo los traduce a 400/404/500	`src/errores.js`, `src/app.js`	La decisión (¿existe?, ¿es válido?) queda en el servicio, que se prueba unitariamente. El controlador queda delgado y se prueba en el nivel de integración.
La raíz de composición es el único lugar donde se crean las piezas reales	`server.js`	Producción usa SQLite y el reloj real. Las pruebas arman la misma app con otras piezas, sin tocar el código.

```
server.js ─► app.js (HTTP) ─► ServicioDeAvistamientos ─┬─► ValidadorDeAvistamiento ─► reloj { hoy() }
                                                       └─► repositorio (SQLite)
```

Un defecto que evitó el reloj. La forma ingenua de obtener "hoy" es `new Date().toISOString().slice(0, 10)`, pero esa expresión devuelve la fecha en UTC. En Colombia (UTC-5), entre las 7 p. m. y la medianoche UTC ya va en el día siguiente, así que la API habría aceptado avistamientos de "mañana". Era un defecto latente que solo se manifiesta de noche. Por eso `RelojDelSistema` calcula la fecha en la zona `America/Bogota`. La captura `11_regla_de_negocio.png` se tomó a las 8:32 p. m. (01:32 UTC del 22) y la API rechaza bien el 2026-10-22.

2. Tabla de casos de la Parte A

La tabla se escribió antes de programar las pruebas, con partición de equivalencia y valores límite (Myers, 1979). En todas las pruebas "hoy" es **2026-10-21**, fijado con `RelojFijo`. Los seis límites obligatorios son los casos 2, 3, 4, 5, 9 y 10.

Especie. Particiones: E1, menos de 3 caracteres (inválida); E2, de 3 a 50 (válida); E3, más de 50 (inválida); E4, no es texto (inválida).

#	Valor	Partición	Tipo	Esperado	Por qué ese valor
1	`"Barranquero"` (11)	E2	Representativo	Válido	Valor típico del tramo válido. Es el camino feliz.
2	`"Ab"` (2)	E1	Límite	400, `especie`	Último valor inválido antes del borde. Detecta un `< 2` en lugar de `< 3`.
3	`"Emú"` (3)	E2	Límite	Válido	Primer valor válido. Detecta el clásico `<=` en lugar de `<`, que rechazaría 3.

#	Valor	Partición	Tipo	Esperado	Por qué ese valor
4	`"A"` × 50	E2	Límite	Válido	Último valor válido. Detecta un `>= 50` que lo rechazaría.
5	`"A"` × 51	E3	Límite	400, `especie`	Primer valor inválido superior. Detecta un `> 51`.
6	`" Ab "`	E1	Límite	400, `especie`	Son 4 caracteres en bruto y 2 reales. Detecta medir la longitud antes de quitar los espacios.
7	`12345`	E4	Inválido	400, `especie`	Tipo equivocado. Sin esta rama, `.trim()` lanzaría una excepción y la API respondería 500 en vez de 400.

Fecha. Particiones: F1, AAAA-MM-DD pasada u hoy (válida); F2, posterior a hoy (inválida); F3, formato distinto (inválida); F4, formato correcto pero fecha inexistente (inválida).

#	Valor	Partición	Tipo	Esperado	Por qué ese valor
8	`2026-09-20`	F1	Representativo	Válido	Fecha pasada típica.
9	`2026-10-21` (hoy)	F1	Límite	Válido	Detecta un `>=` donde va `>`, que rechazaría los avistamientos de hoy.
10	`2026-10-22` (mañana)	F2	Límite	400, `fecha`	Primer día inválido. Detecta comparar contra mañana o dejar un día de holgura.
11	`2027-01-01` con hoy `2026-12-31`	F2	Límite	400, `fecha`	Es el límite que cruza de año. Detecta comparar solo mes y día.
12	`20/09/2026`	F3	Inválido	400, `fecha`	Es el formato más común en Colombia, así que es el error más probable de un usuario.
13	`2026-02-30`	F4	Inválido	400, `fecha`	Pasa la expresión regular. Detecta validar solo el formato y no que la fecha exista.

Lugar y observador. Particiones: L1, con texto (válido); L2, vacío o solo espacios (inválido); L3, ausente (inválido).

#	Valor	Partición	Tipo	Esperado	Por qué ese valor
14	lugar `""`	L2	Inválido	400, `lugar`	Representativo del vacío.
15	observador `" "`	L2	Límite	400, `observador`	Está en el borde entre vacío y con texto. Detecta comprobar `!== ''` sin quitar los espacios.
16	observador ausente	L3	Inválido	400, `observador`	El cliente omite el campo.
17	`{ especie: "Ab", fecha: mañana }`	Varias	Inválido	400 con los 4 campos	La regla pide decir qué campo falló, así que deben aparecer todos y no solo el primero.
18	`["Barranquero"]`	Cuerpo	Inválido	400, `cuerpo`	JSON válido que no es un objeto.

3. Por qué cada nivel

La suite sigue la pirámide (Cohn, 2009): 33 pruebas unitarias, 18 de integración y ninguna de extremo a extremo automatizada.

- **Los límites del validador (casos 1 a 18) son unitarios.** La regla es lógica pura, rápida y determinista gracias al stub. Pasar por HTTP no daría información nueva sobre `3` frente a `2` y sumaría Express y SQLite como posibles causas de una falla. Una unitaria que falla señala la línea; una de integración que falla obliga a depurar varias capas (principio FIRST: rápida e independiente).
- **"El servicio no llama al repositorio cuando los datos son inválidos" es unitaria con spy.** Lo que verifica es una interacción interna: que se valide antes de guardar. Desde HTTP solo se ve el 400. La prueba de integración del POST con datos faltantes verifica el efecto complementario: que la lista siga vacía.
- **El POST tiene pruebas de integración aunque el validador ya esté probado.** La integración detecta lo que la unitaria no puede ver, porque cruza las dos fronteras del sistema, la de HTTP y la de la base de datos:
  - que la ruta existe con el método POST, responde **201** y no 200, y trae el encabezado `Location`;

  - que `express.json()` está montado: si faltara, `req.body` sería `undefined` y toda petición daría 400;
  - que el manejador de errores traduce `ErrorDeValidacion` a 400 con `detalles` en JSON;
  - que el SQL es correcto: si el `INSERT` tuviera `lugar` y `fecha` intercambiados, todas las unitarias pasarían con el fake y solo la integración ("el GET al nuevo id lo devuelve") lo detecta;
  - que el JSON mal formado responde 400, algo que ocurre solo en Express, antes de llegar al servicio.
- **El resumen se prueba solo en integración.** Su lógica está en el SQL (`GROUP BY … ORDER BY cantidad DESC`). Probarlo con el fake sería probar el fake.
- **La prueba del error 500 es de la frontera HTTP.** Usa el Express real con un stub del servicio que lanza un error, porque SQLite en memoria no falla de forma natural.

La independencia (FIRST, la "I") se logra porque cada prueba de integración arma en `beforeEach` una app nueva con una base `:memory:` vacía y prepara sus propios datos. Ninguna depende del orden. La API no habla con servicios externos, así que no hizo falta un servidor simulado (punto 8 del taller).

4. Los dobles

Doble	Tipo	Dónde	Por qué ese tipo	Qué pasaría sin él
`RelojFijo` (`test/dobles`)	**Stub**	Validador, servicio e integración	Solo hace falta que responda una fecha preparada. No verificamos cómo se le llama, así que un mock sobraría.	La prueba de "mañana" escrita con `2026-10-22` pasaría hoy y fallaría desde el 22 de octubre, cuando esa fecha ya es "hoy". Si se calculara con `new Date()`, la prueba repetiría la lógica del código (incluido el defecto de UTC). La prueba no sería repetible.
Función `ahora` en `RelojDelSistema`	**Stub**	`reloj.test.js`	Fija el instante 01:24 UTC del 22 de octubre.	Probar la diferencia de zona horaria obligaría a correr la prueba entre las 7 p. m. y la medianoche.
`RepositorioEnMemoria` (`test/dobles`)	**Fake**	`servicio.test.js`	El servicio necesita un repositorio que funcione: crear y después buscar. Así se verifica el estado ("quedó guardado sin espacios"). Un stub obligaría a preprogramar cada respuesta, y un mock acoplaría la prueba al orden exacto de las llamadas.	El servicio tendría que usar SQLite: la prueba dejaría de ser unitaria y una falla del SQL se confundiría con una del servicio.
`jest.spyOn(repositorio, 'crear' / 'actualizar')`	**Spy**	`servicio.test.js`	Registra si se llamó para revisarlo en el Assert, sin expectativas previas. Es la definición de spy de la Sesión 20.	No habría forma de observar que el servicio no intenta guardar datos inválidos.
`servicioQueFalla`	**Stub**	`error-interno.test.js`	Devuelve una respuesta preparada: lanza un error.	La rama 500 sería imposible de alcanzar en una prueba.
`jest.spyOn(console, 'error')`	**Spy** que además silencia	`error-interno.test.js`	Comprueba que el error quedó registrado y evita ensuciar la salida.	No se sabría si el fallo queda en el log.

No se usó un **mock** a propósito. Preferimos verificar el estado (con el fake) antes que la interacción, porque un mock rompe la prueba con cualquier refactorización interna aunque el comportamiento siga igual. Ninguno de estos dobles sería posible sin la inversión de dependencias: el validador recibe el reloj y el servicio recibe el repositorio, en vez de crearlos con `new`.

5. Lo que no probamos

1. `server.js` **(la raíz de composición).** No tiene prueba automatizada. Solo conecta las piezas, y un error ahí se ve en cuanto se ejecuta `npm start`, como muestran las capturas 01 a 11. Probarlo exige levantar un proceso real en un puerto: es una prueba lenta y frágil, del nivel más alto de la pirámide. Se excluye de la cobertura con `collectCoverageFrom: ["src/**/*.js"]`.
2. **Pruebas de extremo a extremo con el reloj real.** Las capturas con `curl` contra el servidor son una verificación manual. No se automatizaron porque su resultado depende del día en que se ejecutan (no serían repetibles) y porque la combinación reloj real más zona horaria ya está cubierta en el nivel unitario con el stub de `ahora`.
3. **Concurrencia, como dos `PUT` simultáneos sobre el mismo id.** La API usa SQLite embebido en un solo proceso. Por el principio "las pruebas dependen del contexto", el riesgo es bajo y la prueba sería no determinista.
4. **Especies con emojis o caracteres compuestos.** `String.length` cuenta unidades UTF-16, así que un emoji cuenta como 2. Los nombres de especies son alfabéticos y lo aceptamos como riesgo conocido. No se escribió una prueba porque implicaría cambiar la regla (contar grafemas), y eso sería una decisión de validación con el profesor, no de verificación.

6. Lectura de la cobertura

La herramienta es `npm run cobertura` (Jest con istanbul). El reporte HTML queda en `coverage/index.html` y hay capturas en `capturas/14` a `capturas/17`. Se miraron cuatro criterios de cobertura: sentencias, ramas, funciones y líneas.

Primera lectura y lectura final (los demás archivos, `db`, `errores`, `servicio` y `validador`, quedaron en 100 % en todos los criterios):

```
Archivo          Antes: % Stmts / Branch / Lines    Final: % Stmts / Branch / Lines    Sin cubrir
All files               97.52 / 91.48 / 97.29              99.17 / 93.61 / 99.09
app.js                  93.54 / 83.33 / 93.54             100    / 100   / 100       (57–58 cubiertas)
reloj.js                88.88 / 33.33 / 85.71              88.88 / 33.33 / 85.71       5
repositorio.js         100    / 50    / 100               100    / 50    / 100        27
```

Línea o rama sin cubrir	¿Hace falta prueba?	Razón
`app.js` 57-58: la rama del **500**	**Sí, y se escribió** (`error-interno.test.js`)	Es el contrato de que un fallo inesperado responde JSON y **no expone el detalle interno**, en este caso el mensaje de SQLite. Es un riesgo de seguridad y de contrato de la API, y la prueba es barata con un stub.
`reloj.js` 5: la función por defecto `() => new Date()` y el `= {}` (2 de 3 ramas)	No vale la pena	El comportamiento con riesgo, la zona horaria, sí está cubierto con el stub de `ahora`. Lo que falta es solo `new Date()`, que es código de Node. Probarlo sería probar la plataforma o escribir una prueba que depende de la hora. Además se ejecuta cada vez que se levanta el servidor.
`repositorio.js` 27: la rama `: undefined` de `actualizar`	No vale la pena	Solo se alcanza si el registro se borra entre la comprobación de existencia del servicio y el `UPDATE`, una condición de carrera imposible con un proceso y SQLite embebido. Es código defensivo. Si el repositorio se reutilizara sin el servicio, habría que probarlo directamente.

Conclusión. La cobertura dice qué se ejecutó, no qué se verificó: una prueba sin `expect` también daría 99 %. Por eso cada prueba de esta suite verifica el cuerpo y el efecto, no solo el código de estado. El criterio de cobertura de ramas fue más útil que el de líneas: `repositorio.js` tiene 100 % de líneas y aun así 50 % de ramas, y solo la cobertura de ramas mostró que el `: undefined` nunca se ejecuta. No buscamos el 100 %: tres ramas, en dos archivos, quedan sin cubrir a propósito y con razón.

Uso de IA

Se usó Claude (Anthropic), modelo Claude Opus 5.5, como apoyo para refactorizar, escribir las pruebas y redactar este documento. Las decisiones están explicadas aquí, y el código fue revisado, ejecutado y entendido por el autor.